vw.arista.com/en/) Login (https://arista.my.site.com/AristaCommunity/s/login/)

ROUTING/SWITCHING (/ARISTACOMMUNITY/S/TOPIC/0…

Title
Understanding EOS CLI implementation

Article Type
Tech Tips

Author
Edgar Shine (/AristaCommunity/s/profile/005F0000002hTwIIAE)

Published Date
October 31, 2013

Updated Date
October 4, 2024

Table of Contents

Content

Understanding EOS CLI implementation

The document is not applied to EOS4.25.2f and later releases. Please refer to the following link for newer EOS:
https://www.arista.com/en/support/toi/eos-4-25-2f/14712-cli-extensions-for-customers (https://www.arista.com/en/support/toi/eos-4-25-2f/14712-cli-extensions-for-customers).
This document gives an overview of the some of the APIs used for developing CLI Plugins. Although it tries to be as complete as possible, it only focuses on the most common APIs used by the CLI Plugins and it is not an exhaustive description of all the interfaces. The document is relevant for EOS version 4.12.0, although parts of the document might also apply to older releases.

“Hello world!” example

TestCli.py

```
# Copyright(c)2013 AristaNetworks,Inc. All rights reserved.
# Arista Networks, Inc. Confidential and Proprietary.
import BasicCli
import CliParser
from CliModel import Model
# Model  only required for eAPI support (starting 4.12.0)
class HelloMessage( Model ):
    def render( self ):
        print 'Hello world!'
# Behaviour
def sayHello( mode ):
    # print 'Hello world!' # no eAPI support
    return HelloMessage() # eAPI support
# Rule definition
tokenHello = CliParser.KeywordRule( 'hello',
                                    helpdesc='Print "Hello world!"' )
# Command definition
BasicCli.registerShowCommand( tokenHello, sayHello )
# Plugin definition
def Plugin( entityManager ):
    pass
```

Installation

```
Arista(bash)%cp TestCli.py /usr/lib/python2.7/sitepackages/CliPlugin
Arista(bash)%Cli
Arista>en
Arista#show hello
Hello world!
```

Cli Registration

A CLI plugin defines four kinds of objects:

1. **rule**: object that expresses the syntax of a particular CLI command, and its binding to an action.
2. **action**: action that is executed when a CLI command is entered by the user (or read from a configuration file).
3. **mode**: object that represents a new CLI mode (e.g enable mode, config mode or config-if mode for interface Ethernet0/1). A user's CLI session is always in a single mode at any given time, indicated by the prompt (e.g. localhost(config-vlan)#).
4. **modelet**: object that extends an existing CLI mode to add extra rules and state.

```
BasicCli.registerShowCommand( *args, **kwargs )
```

where args = (<rule>, [<rule>, ...], <action>), kargs = hidden | privileged | priority

Registers a show command that will be present in Enabled and Unprivileged modes.

Key arguments:

- **privileged**: if specified, command will not be present in Unprivileged mode
- **hidden**: if specified, command will be hidden to the users and will not provide any completions

e.g.

```
BasicCli.registerShowCommand( tokenShowAaa, showAaa, privileged=True )
BasicCli.registerShowCommand( tokenElephants, showElephants, hidden=True )
```

Commands registered with this function automatically support output filtering , such as:

```
Arista#show ... | include blah
Arista#show ... | redirect myfile
```

```
BasicCli.registerShowCommandWithMode( mode, *args, **kwargs )
```

where: args = (, [, ...],) kargs = hidden | privileged | priority

Register a show command with the specific mode. args and kwargs are the same as above.

e.g.

```
(StpCli.py) BasicCli.registerShowCommandWithMode( MstConfigMode, CliParser.Op
```

Basic modes:

BasicCli.EnableMode
BasicCli.UnprivMode
BasicCli.GlobalConfigMode

Other modes:

AaaCli.RoleConfigMode
AclCli.BaseAclConfigMode
AclCli.CpConfigMode
ArthostCli.ArthostConfigMode
ArthostCli.ArthostGroupConfigMode
Capi.CapiVrfConfigMode
Capi.CapiVrfConfigModeBase
CliCli.AliasConfigMode
ConfigMgmtMode.ConfigMgmtMode
DiagCli.SuiteConfigMode
EmailCli.EmailConfigMode
EventCli.EventHandlerConfigMode
FocalPointV2LoadBalance.ConfigLoadBalancePoliciesMode
FocalPointV2LoadBalance.ConfigLoadBalanceProfileMode
FsfCli.FsfConfigMode
IgmpProfileCli.igmpProfileConfigMode
IntfCli.IntfConfigMode
IntfRangeCli.IntfRangeConfigMode
IraVrfCli.VrfAddressFamilyMode
IraVrfCli.VrfDefinitionMode
LanzCli.LanzStreamingMode
LauncherDaemonCli.DaemonConfigMode
LinkFlapCli.LinkFlapConfigMode
MlagConfigCli.ConfigMlagMode
NatCli.NatPoolConfigMode
QosCli.ClassMapMode

QosCli.IntfTxQueueConfigMode
QosCli.PolicyMapClassMode
QosCli.PolicyMapMode
RedSupCli.RedundancyMode
RouteMapCli.Ipv6PrefixListMode
RouteMapCli.RouteMapMode
RoutingBgpCli.RouterBgpAfMode
RoutingBgpCli.RouterBgpMode
RoutingIsisCli.RouterIsisAfMode
RoutingIsisCli.RouterIsisMode
RoutingOspf3Cli.RouterOspf3Mode
RoutingOspfCli.RouterOspfMode
RoutingRipCli.RouterRipMode
SandCli.DistributedHostsConfigMode
Ssh.SshVrfConfigMode
StpCli.MstConfigMode
StrataMmuCli.MmuQueueConfigMode
TapAggIntfCli.TapAggConfigMode
Telnet.TelnetVrfConfigMode
UplinkFailureDetectionCli.LinkStateConfigMode
VlanCli.VlanConfigMode
Vm.VmConfigMode
VmTracerCli.VCenterConfigMode
VmTracerCli.VShieldConfigMode

```
<mode>.addCommand( ( , [ <rule, ...> ], ) )
```

A command is a rule that defines one complete command that a user would type into the Cli. The function of such a rule typically performs some action, such as modifying Sysdb state in the case of a config command, or displaying Sysdb state in the case of a show command.

e.g.

```
(BasicCli.EnableMode.addCommand( ( tokenDiagnostic, gotoDiagMode ) )
```

Rule definition

Rule types

- static match: TokenRule, with subclasses:
  - KeywordRule
  - PatternRule
  - RangeRule

    - FloatRangeRule
- dynamic match:
    - DynamicKeywordsMatcher
- concatenation of rules: ConcatRule
- alternation of rules: OrRule
    - DynamicNameRule
- wrapper rules: WrapperRule, with subclasses:
    - OptionalRule
    - HiddenRule
- iteration rules: IterationRule, with subclasses:
    - StringRule

Static match

```
CliParser.TokenRule( matcher, [ value, guard, ... ] )
```

Rule that matches a single whitespace-delimited token.

e.g.

```
    tokenIntfName = CliParser.TokenRule( matcher=intfNameMatcher, guard=guard )
```

When a TokenRule matches, it returns a value. The value of a TokenRule can be changed by passing a value function as the value parameter to the rule's constructor. This value function is called when the TokenRule matches, and is passed two parameters:

1. **mode**, the current mode
2. **match**, for a KeywordRule or a RangeRule the default value, the matching token. However, for a PatternRule this is the match object returned from the call to re.match(token,pattern). See the Python documentation (http://docs.python.org/2/library/re.html#match-objects) for more information on match objects.

   The result returned from the value function becomes the value of the TokenRule. Note that value functions should return quickly and should not have any sideeffects, as they may be called in the middle of parsing, or when a user types or "?". In particular, value functions should not raise any exceptions.

   e.g.

```
tokenHostTraps = CliParser.KeywordRule( 'traps',
                                        helpdesc='Send Trap messages to this
                                        value=lambda mode, match: 'trap' )
```

The **matcher** parameter is expected to be an object that implements two functions:

- **match**: takes a token (of type string) and returns either:
  - *None* if the token does not match
  - the value of the match if the token does match, where the type of the value is defined by the specific matcher.
- **completions**: which takes a partial token (of type string) and returns a list of Completion objects representing the possible matching completions of that partial token.

e.g.

```
class KeywordMatcher( object ):
    """Type of matcher that matches a single keyword. The match is
    caseinsensitive and it returns the original keyword, casepreserved.
    This improves performance significantly, for example, in the case of
    loading startupconfig with a lot of names."""
    __slots__ = ( 'keyword_', 'completion_', 'alternates_' )
    def __init__( self, keyword, helpdesc, alternates=None ):
        self.keyword_ = keyword
        self.completion_ = Completion( keyword, helpdesc )
        self.alternates_ = tuple( alternates ) if alternates else ()
def match( self, mode, token ):
    token = token.lower()
    if token == self.keyword_.lower():
        return self.keyword_
    for k in self.alternates_:
        if token == k.lower():
            session = mode.session_
            return self.keyword_
    return None
def completions( self, mode, token ):
    if self.keyword_.lower( ).startswith( token.lower() ):
    return [ self.completion_ ]
    return []
```

A token rule may have a guard, which is a function that takes a mode and a token, and returns:

- None (if we should go ahead and let the token through)

- a guard code if the user should not be allowed to use this token at this time.

  Standard guard codes:

  CliRule.guardNotThisEosVersion
  CliRule.guardNotThisPlatform
  CliRule.guardNoLicense
  CliRule.guardNotThisSupervisor

  e.g.

```
    def vlanMappingSupportedGuard( mode, token ):
        if bridgingHwCapabilities.vlanXlateSupported:
            return None
        else:
            return CliParser.guardNotThisPlatform
```

```
CliParser.KeywordRule( keyword, helpdesc, [ alternates, ... ] )
```

  Rule that matches a single (caseinsensitive) keyword.

  e.g.

```
    tokenHello = CliParser.KeywordRule( 'hello',
                                        helpdesc='Print "Hello world!"' )
```

  The matcher is caseinsensitive and it returns the original keyword, casepreserved. This improves performance significantly, for example, in the case of loading startup-config with a lot of names.

  helpdesc [and **helpname**] are used when printing interactive help to the user when they type **<tab>** or **?**. However, **helpname** should not be specified for a *KeywordRule* or for a *RangeRule* (see below), as these have sensible defaults.

  You may pass in **alternates** as a set. The keyword rule will recognize an alternate as an exact match (no completion required). No help is displayed for alternates, and they do not affect completions. For example, this is useful so that "**show interface stat**" can run "**show interface stats**" even though there is another command "**show interface status**". As another example, "**show boot**" should be the same as "**show bootconfig**" even though there is another command "**show bootextensions**". To make this happen, the "**bootconfig**" keyword rule specifies "**boot**" as an alternate.

  e.g.

```
      tokenBootConfigAfterShow = CliParser.KeywordRule( 'bootconfig',
                                                        alternates=set(('boot',)),
                                                        helpdesc='Show boot configu
```

CliParser.PatternRule(pattern, helpname, helpdesc, ...)

Rule that matches a (case-sensitive) regular expression pattern. The value of it is the matching token itself.

e.g.

```
      groupNameRe = '[^\\s]+'
      tokenGroupName = CliParser.PatternRule( groupNameRe,
                                              name='groupName',
                                              helpname='WORD',
                                              helpdesc='Name of the group' )
```

CliParser.RangeRule(lbound, ubound, helpdesc, [...])

Rule that matches an integer between 'lbound' and 'ubound' (inclusive). The value of it the matching token converted to an int.

e.g.

```
      vlanId = CliParser.RangeRule( 1, 4096, helpdesc='VLAN ID', name='vlanId' )
```

CliParser.FloatRangeRule(lbound, ubound, helpdesc, [...])

Rule that matches a float between 'lbound' and 'ubound' (inclusive).

e.g.

```
      level = CliParser.FloatRangeRule( 0.01, 100,
                                        helpdesc='Maximum bandwidth percentage' \\
                                                 ' allowed by storm control',
                                        name='level', precisionString = '%.5g' )
```

Dynamic match

CliParser.DynamicKeywordsMatcher(keywordsFn, emptyTokenCompletion, ...)

The DynamicKeywordsMatcher is a special matcher that allows dynamic keyword matching and special completion handling.

The **keywordsFn** parameter is a callback function with a single argument, the current mode. Its return value depends on the value of **emptyTokenCompletions**, which specifies the completion for an empty token (where the user types '?' without a partial token, and displays available tokens and help descriptions):

- If emptyTokenCompletions is *None*, keywordsFn should return a dictionary of keywords, mapping keywords to help descriptions.
- If emptyTokenCompletions is not *None*, keywordsFn may return any iterable object (e.g. a dict, list, tuple, set or Sysdb collection) whose iterator returns a sequence of keywords.

One obvious use case is to generate keywords dynamically. For example, you may have a set of keywords for a show command, which may be dependent on platform.

A common case of keywordsFn is just to return an existing dictionary:

```
lambda mode: some_dict
```

The **emptyTokenCompletion** parameter allows a generic Completion list to be returned for completing an empty token. This is useful when we do not want the keywords to clutter the help screen, but at the same time we want to be able to complete partial tokens. One such example is the MonthRule. For now, this usage pattern requires literal=False[2] for **emptyTokenCompletion**.

For example, the following implements a MonthRule:

```
monthMap = { 'January':1, 'February':2, 'March':3,
             'April':4, 'May':5, 'June':6,
             'July':7, 'August':8, 'September':9,
             'October':10, 'November':11, 'December':12 }
monthMatcher = CliParser.DynamicKeywordsMatcher(
         # helpdesc doesn't matter when using emptyTokenCompletion
         # so we can return monthMap directly
         lambda mode: monthMap,
         emptyTokenCompletion=[ CliParser.Completion(
              'MONTH', 'Month of the year (Jan, Feb, etc.)',
               literal=False ) ] )
monthRule = CliParser.TokenRule( matcher=monthMatcher, name='month',
                                 value=lambda mode, match: monthMap[ match ]
```

The behavior is:

```
Arista#clock set 1:2:3 ?
  <1-31>      Day of the month
  MONTH       Month of the year (Jan, Feb, etc.)
  mm/dd/yyyy  Today's date
Arista#clock set 1:2:3 j?
  January July June
Arista#clock set 1:2:3 ja<tab>
Arista#clock set 1:2:3 january
```

Note there is only one "MONTH" help description when the user enters '?' without partial token, but if the user enters '?' after a partial token, it shows all valid completions for the months.

Please be aware that DynamicKeywordsMatcher is caseinsensitive just like KeywordRule. If you have a potentially large number of keywords, it may have performance issues if used in a config command (imagine all these commands are in the startupconfig). For these reasons, the DynamicNameRule is a much better tool for matching user created names.

Concatenation of rules

```
CliParser.ConcatRule( subrules, ... )
```

Rule that matches the concatenation of a sequence of rules. ConcatRules are rarely constructed directly; they are usually constructed from rule expressions. The default value of a ConcatRule is the value of the rightmost subrule.

e.g.

```
CliParser.ConcatRule( tokenHost, tokenFilterHostname, name='host' )
```

Alternative notation:

```
( name, rules, value )
```

where:

- value: function or lambda expression; sets the value function, optional
- name: string beginning with '>>'; sets the name; optional

e.g.

```
    hostVrfRule = ( tokenHostVrf, vrfNameRule )
```

Alternation of rules

```
CliParser.OrRule( subrules, ... )
```

Rule that matches precisely one of a set of rules. The default value of an OrRule is the value of whichever alternative matched.

e.g.

```
    firstSecondOrThird = CliParser.OrRule( firstRule, secondRule, thirdRule )
```

Alternative notation:

```
    firstSecondOrThird = CliParser.OrRule()
    firstSecondOrThird |= firstRule
    firstSecondOrThird |= secondRule
    firstSecondOrThird |= thirdRule
```

If multiple subrules match a user input, the OrRule gets confused and throws a GrammarError. When this happens, it always means the rules are not constructed properly. Consider the following example:

```
    serverGroupRule = CliParser.PatternRule( '[AZaz09_]+',
                                             helpname='WORD',
                                             helpdesc='Servergroup name' )
    tacacsGroupRule = CliParser.KeywordRule( 'tacacs+', helpdesc='all tacacs+ ser
    radiusGroupRule = CliParser.KeywordRule( 'radius', helpdesc='all radius serve
    orRule = CliParser.OrRule( serverGroupRule, tacacsGroupRule, radiusGroupRule
```

The problem with the above rule is that if the user enters 'radius', then there are two rules that would match, confusing the OrRule.

To solve this problem, the concept of **priority** was introduced. If an OrRule finds multiple matches, it picks the rule with the highest priority (the lowest priority value). Of course, if there are still multiple rules left, then it would still throw a GrammarError. There are currently three priorities defined:

- PRIO_HIGH
- PRIO_NORMAL

- PRIO_LOW

By default KeywordRule has PRIO_HIGH, and all other rules have PRIO_NORMAL, so in the above case, if the user enters 'tacacs+' or 'radius', the corresponding KeywordRule would match instead of the generic PatternRule.

```
CliParser.DynamicNameRule( namesFn, ... )
```

DynamicNameRule is a type of OrRule which behaves the same way as a PatternRule with the pattern, helpname and helpdesc parameters, but also provides auto-completion on names returned by the **namesFn** function parameter. It is intended to be used in places where we expect object names (ACL, host group, port profile, anything that has a name).

The namesFn could return any iterable collection of names (dict, list, etc). In the below example it return the Sysdb collection (ipAclConfig is an instantiating collection of ACLs indexed by name). Unlike DynamicKeywordsMatcher, names are case-sensitive (e.g. users can specify 'abc' and 'ABC' as two different names).

One example of using it for ACL names:

```
    aclConfig = LazyMount.mount( 'security/acl/config', 'Acl::Config', 'w' ) ...
    aclNameRule = CliParser.DynamicNameRule( lambda mode: aclConfig.ipAclConfig,
                                             'Accesslist name' )
```

The behavior is:

```
    Arista#show ip accesslists ?
      WORD Accesslist name
      | Output modifiers
    Arista#show ip accesslists d?
      WORD defaultcontrolplaneacl
    Arista#show ip accesslists d<tab>
    Arista#show ip accesslists defaultcontrolplaneacl
```

Wrapper rules

```
WrapperRule( subrule, [ guard, ... ] )
```

Rule that wraps another rule, delegating all operations to the wrapped rule.
The guard function works similarly to TokenRule. You can use WrapperRule to guard any rule by wrapping it.

e.g.

```
    tokenNoOrDefault = CliParser.WrapperRule( BasicCli.noOrDefault, name='no' )
```

HiddenRule

Rule that behaves exactly like an underlying rule but provides no completions.

e.g.

```
    scpRule = ( tokenScp, scpCmdLine, doScpServer )
    BasicCli.EnableMode.addCommand( CliParser.HiddenRule( scpRule )
```

OptionalRule(optionalRule, ...)

Type of WrapperRule that accepts an underlying rule zero or one times. The default value of an OptionalRule is the value of its subrule, if the subrule matched, or None otherwise.

e.g.

```
    vlanIdOpt = CliParser.OptionalRule( vlanId )
```

Alternative notation:

```
    [ name, rule, value ]
```

where:

- value: function or lambda expression; sets the value function, optional
- name: string beginning with '>>'; sets the name; optional

e.g.

```
    vlanIdOpt=[ vlanId]
```

SetRule(subrules, [mandatory, exclusive, minMembers])

Type of OptionalRule that accepts any subset (without duplicates) in any order of a set of underlying rules.

e.g.

```
tokenA = KeywordRule( 'a', name='a') tokenB = KeywordRule( 'b' )
 tokenC = KeywordRule( 'c' )
 s = SetRule( tokenA, tokenB, tokenC )
```

When **s** is fed input “c b a” it produces this result:

```
[(None,'c')
,(None,'b')
,('a','a')]
```

It can be useful to have **unnamed members** of a SetRule if the individual results are fully identified by their value, as might be the case for a set of keyword alternatives, for instance.

Optional parameters:

- **mandatory**: a tuple of mandatory rules that have to match once. In the following example, tokenA is mandatory, so 'b a', 'a c', 'c b a' would all match, but 'b c' would not.

```
SetRule( tokenA, tokenB, tokenC, mandatory=( tokenA, ) )
```

- **exclusive**: a tuple of tuples that specifies mutually exclusive rule sets. In the following example, tokenA and tokenB are mutually exclusive, and tokenA and tokenC are also mutually exclusive, so 'b c' would match, but 'b a' or 'a c' would not.

```
SetRule( tokenA, tokenB, tokenC, exclusive=( ( tokenA, tokenB ), ( tokenA
```

Note: Be aware that if two rules are mutually exclusive, and both inhale a common set of tokens with one of the rule being ready to end, then the other will never be able to match. For example, ConcatRule(tokenA, tokenB) and ConcatRule(token A, [tokenC]).

- **minMembers**: restricts SetRule to accept at least a minimum count of subrules. For example:

```
SetRule( a, b, c, minMembers=2 )
```

'a b', 'b c', 'a c', 'a b c' would match, but '', 'a', 'b', 'c' would not.

Iteration rules

```
IterationRule( iteratedrule, [ ... ] )
```

Rule that accepts an underlying rule one or more times. The default value of an IterationRule is a list containing the values of the subrule each time it matched.

e.g.

```
agentName = CliParser.PatternRule( ".+", helpname='WORD', helpdesc='Agent nam
agentNameList = CliParser.IterationRule( agentName )
```

StringRule(...)

Type of IterationRule that matches an arbitrary string.

e.g.

```
descriptionRule = CliParser.StringRule( helpname='description',
                                        helpdesc="Test description",
                                        name="description")
```

Standard tokens

BasicCli.tokenDefault
BasicCli.tokenNo
BasicCli.noOrDefault
BasicCli.trailingGarbage: useful for allowing a no command to accept optional parameters

Others:

IraCli.Intf.rule
MacAddr.macAddr HostnameCli.HostnameRule etc.

Common Cli tokens

CliToken is a mechanism to share common Cli rules (such as the ip keyword) across packages.
See: /usr/lib/python2.7/sitepackages/CliToken for the complete list of available tokens.

e.g.

```
BasicCli.EnableMode.addCommand( ( CliToken.Clear.clear, CliToken.Ip.ipForClea
                                  tokenAccessLists, ... ) )
```

Note, the same token could be used in different command contexts with slightly different help description, so you probably want to define a new one for each context.

Behaviour

An action is the function that is called when a complete CLI command is executed, and carries out the command. An action is really just the value function for the toplevel ConcatRule for the command, but actions differ qualitatively from other value functions:

- they don't return anything
- they have no restriction on sideeffects
- they are subject to authorization checks

Routing/Switching (/AristaCommunity/s/topic/0TO2I00…

EOS/cEOS/vEOS (/AristaCommunity/s/topic/0TO2I00…

Files (0) (/AristaCommunity/s/relatedlist/ka0Uw0000005fNVIAY/AttachedContentDocuments)

Related Articles

Installing/Uninstalling a software patch on an Arista device (/AristaCommunity/s/article/installing-uninstalling-a-software-patch) 3.23K

Understanding EOS Software Download Options (/AristaCommunity/s/article/understanding-eos-software-download-options) 6.89K

Memory utilization on EOS devices (/AristaCommunity/s/article/memory-utilization-on-eos-devices) 15.25K

Understanding CPU Utilization (/AristaCommunity/s/article/understanding-cpu-utilization) 11.17K

Arista EOS – BGP Selective Route Download (/AristaCommunity/s/article/arista-eos-bgp-selective-route-download) 5.11K

Trending Articles

High Availability edge upgrade fails and does not retry
(/AristaCommunity/s/article/High-Availability-edge-upgrade-fails-and-does-not-retry)

Verbose counters on Arista 7130 series
(/AristaCommunity/s/article/verbose-counters-on-arista-7130-series)

Internet BGP peering examples for enterprises
(/AristaCommunity/s/article/internet-bgp-peering-examples-for-enterprises)

Troubleshooting Parity Errors
(/AristaCommunity/s/article/Troubleshooting-Parity-Errors)

VMWare NSX-T 3.0 EVPN Type 5 Integration with Arista Gateways
(/AristaCommunity/s/article/vmware-nsx-t-3-0-evpn-type-5-integration-with-arista-gateways)

Get In Touch Today

Contact Us (https://www.arista.com/en/company/contact-us)

ARISTA

(https://www.facebook.com/AristaNW) (https://twitter.com/@AristaNetworks)
(https://www.linkedin.com/company/arista-networks-inc)

Support

Support & Services
(https://www.arista.com/en/support/customer-support)

Training
(https://www.arista.com/en/partner/partner-portal/training)

Product Documentation
(https://www.arista.com/en/support/product-documentation)

Contacts & Help

Contact Arista
(https://www.arista.com/en/company/contact-us)

Contact Technical Support
(https://www.arista.com/en/support/customer-support)

Order Status (https://orders.arista.com/)

Software Downloads (https://www.arista.com/en/support/software-download)

News

News Room (https://www.arista.com/en/company/news/in-the-news)

Events (https://www.arista.com/en/company/news/events)

Blogs (https://www.arista.com/blogs)

About Arista

Company (https://www.arista.com/en/company/company-overview)

Management Team (https://www.arista.com/en/company/management team)

Careers (https://www.arista.com/en/careers)

Investor Relations (https://investors.arista.com/)

© 2026 Arista Networks, Inc. All rights reserved.
Terms of Use (https://www.arista.com/en/terms-of-use) Privacy Policy (https://www.arista.com/en/privacy-policy)